Trabalho Prático 1 — Mecanismos de IPC

Sistemas Distribuídos — CEFET-MG

Pedro Henrique Ferreira Santos e Thiago Leonardo Oliveira Bertolino

2026/1

O código-fonte completo da implementação está disponível em:
https://github.com/pedrenrique/sistemas-distribuidos/

Pipes

1.1 Implementação

Nesta parte do trabalho foi implementado o problema Produtor-Consumidor utilizando dois processos que se comunicam por meio de um pipe anônimo em linguagem C.

Primeiro, o programa cria um pipe, que serve como um canal de comunicação entre processos. Em seguida, o `fork()` é chamado e o programa passa a ter dois fluxos de execução: o processo pai, que atua como produtor, e o processo filho, que atua como consumidor.

Uma decisão importante na implementação foi o uso de mensagens com tamanho fixo de 20 bytes. Como o pipe trabalha com fluxo contínuo de bytes, essa padronização evita problemas de leitura parcial e garante que cada leitura no consumidor corresponda exatamente a uma mensagem enviada pelo produtor. Além disso, os números inteiros são convertidos para string antes de serem enviados e depois reconvertidos no consumidor, o que simplifica a comunicação entre os processos.

O produtor é responsável por gerar uma sequência de números inteiros começando em 1. A cada novo valor, é somado um número aleatório entre 1 e 100, fazendo com que a sequência cresça de forma variável. Esses números são enviados pelo pipe já convertidos para texto. Ao final da geração, o produtor envia o valor 0, que foi definido como sinal de término da execução.

O consumidor fica responsável por ler os dados que chegam pelo pipe. Cada valor recebido é convertido novamente para inteiro e verificado para saber se é primo. O resultado dessa verificação é exibido no terminal. Quando o valor 0 é recebido, o consumidor entende que não há mais dados e encerra sua execução.

A comunicação entre os processos acontece de forma sequencial através do pipe, o que faz com que o consumidor aguarde automaticamente quando não há dados disponíveis.

Dessa forma, não foi necessário implementar mecanismos adicionais de sincronização entre produtor e consumidor. Por fim, o processo principal utiliza `waitpid` para aguardar o término do processo filho antes de encerrar o programa, garantindo que toda a execução ocorra de forma correta e ordenada.

1.2 Testes e Resultados

Para validar o funcionamento do programa, foram realizados três testes variando a quantidade de números gerados pelo produtor. O objetivo foi verificar a correta comunicação entre os processos, a integridade dos dados transmitidos e o comportamento do consumidor na verificação de números primos.

Teste 1 — 10 números

O primeiro teste foi realizado com 10 números gerados pelo produtor. Esse caso foi utilizado para verificar o funcionamento básico do programa e facilitar a observação da saída no terminal.

```
[Produtor] vai gerar 10 numeros
[Produtor] enviando: 1
[Produtor] enviando: 13
[Produtor] enviando: 55
[Produtor] enviando: 114
[Produtor] enviando: 205
[Produtor] enviando: 239
[Produtor] enviando: 302
[Produtor] enviando: 397
[Consumidor] esperando numeros...
[Produtor] enviando: 432
[Produtor] enviando: 514
[Produtor] terminou
[Consumidor] #1 -> 1: nao primo
[Consumidor] #2 -> 13: PRIMO
[Consumidor] #3 -> 55: nao primo
[Consumidor] #4 -> 114: nao primo
[Consumidor] #5 -> 205: nao primo
[Consumidor] #6 -> 239: PRIMO
[Consumidor] #7 -> 302: nao primo
[Consumidor] #8 -> 397: PRIMO
[Consumidor] #9 -> 432: nao primo
[Consumidor] #10 -> 514: nao primo
[Consumidor] recebeu sinal de fim
[Consumidor] total processado: 10
[Main] programa finalizado
```

Figura 1: Saída do terminal para o Teste 1 (10 números).

Durante a execução, foi possível observar que o produtor gerou e enviou os valores corretamente, enquanto o consumidor recebeu os dados na ordem esperada e realizou a verificação de primalidade de cada número. O valor 0 foi corretamente interpretado como sinal de término, encerrando o consumidor após o processamento de todos os elementos.

Para os testes 2 e 3, com 100 e 1000 valores respectivamente, o comportamento do sistema permaneceu consistente com o observado no primeiro teste. Para evitar excesso de conteúdo no relatório, as saídas completas desses testes estão disponíveis no repositório do GitHub do projeto.

Produtor-Consumidor com Semáforos

2.1 Parâmetros do Programa

O programa é controlado por três parâmetros fundamentais: N, N_p e N_c.

N — **tamanho do buffer compartilhado.** É o número de posições disponíveis no vetor de memória compartilhada. Aumentar N reduz o número de bloqueios causados por buffer cheio ou vazio, mas não elimina o gargalo quando há desbalanceamento severo entre threads.

N_p — **número de threads produtoras.** Cada thread produtora gera números aleatórios entre 1 e 10^7 e os insere no buffer. Aumentar N_p eleva a taxa de produção, mas também aumenta a contenção nos semáforos de sincronização.

N_c — **número de threads consumidoras.** Cada thread consumidora retira um número do buffer, verifica se é primo e imprime o resultado.

Em resumo, o desempenho do sistema depende do equilíbrio entre os três parâmetros: N define a "folga" de armazenamento, enquanto N_p e N_c definem as taxas de produção e consumo. O ponto ótimo ocorre quando as taxas são próximas e o buffer é grande o suficiente para absorver variações naturais do escalonamento do sistema operacional.

2.2 Decisões de Projeto e Implementação

O programa foi implementado em C com pthreads, usando um vetor circular de inteiros de tamanho N como memória compartilhada, sincronizado por três semáforos POSIX: `sem_vazio` (inicializado com N), `sem_cheio` (inicializado com 0) e `mutex_buf` (semáforo binário). Produtores executam `wait(sem_vazio)` antes de inserir e `signal(sem_cheio)` após; consumidores fazem o inverso — garantindo bloqueio automático quando o buffer estiver cheio ou vazio.

Cada produtora usa `rand_r` com semente única por thread. A verificação de primalidade ocorre fora da região crítica para não bloquear outras threads. O programa encerra após $M = 10^5$ itens consumidos, controlado por contador global protegido por mutex, e a ocupação do buffer é registrada após cada operação.

2.3 Estudo de Caso — Tempo de Execução

Para cada combinação (N, N_p, N_c), o programa foi executado 10 vezes e o tempo médio calculado. $N = 1$ é muito mais lento (3,7 s a 6,9 s) pois força execução quase serializada. Para $N \geq 10$, os tempos convergem (0,1 s a 2,0 s) e o desempenho passa a depender do balanceamento entre N_p e N_c: a combinação $(2, 1)$ foi a mais eficiente,

enquanto grandes assimetrias como $(1, 8)$ ou $(8, 1)$ aumentam o *overhead* de sincronização sem ganho de *throughput*.

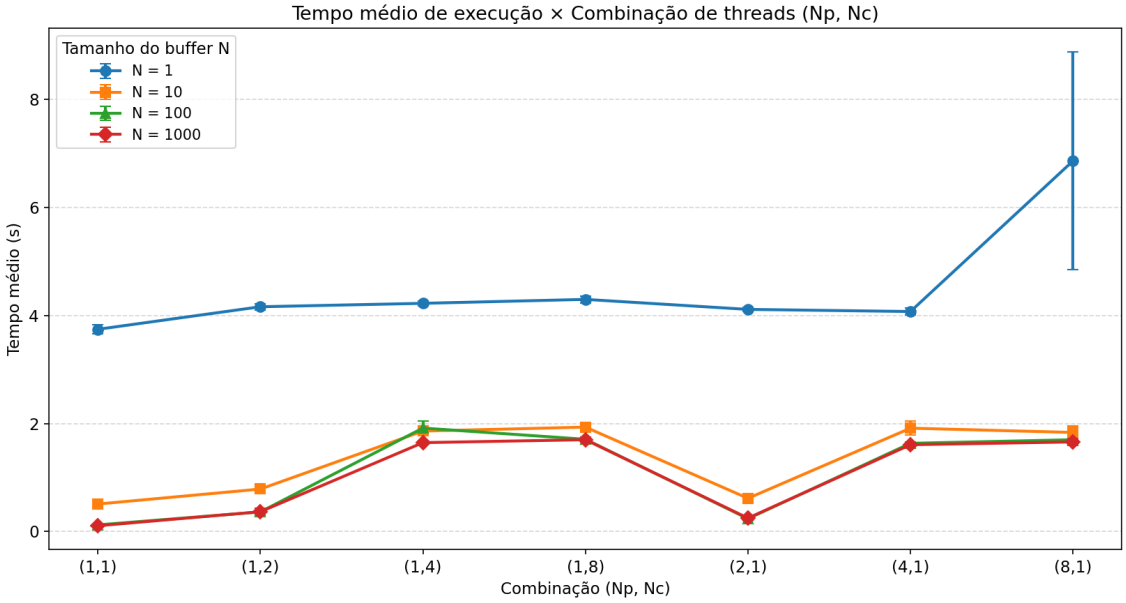
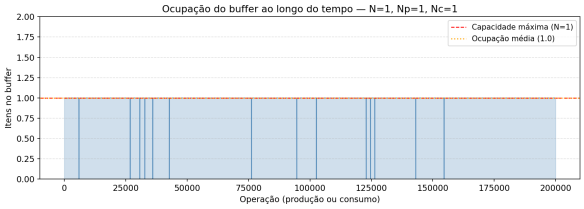


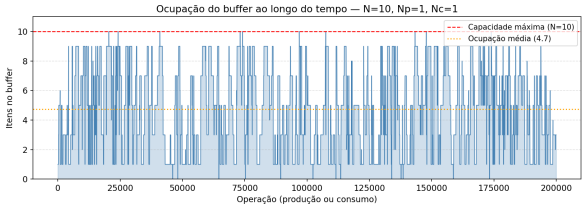
Figura 2: Tempo médio de execução em função da combinação (N_p, N_c) para cada N.

2.4 Estudo de Caso — Ocupação do Buffer

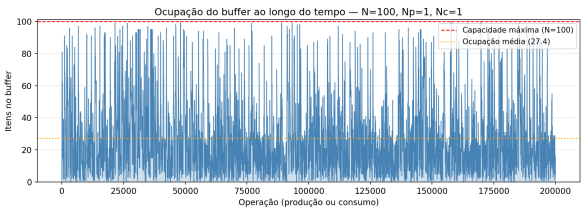
Os gráficos abaixo mostram a ocupação do buffer com $(N_p = 1, N_c = 1)$ para cada valor de N, isolando o efeito do tamanho do buffer. À medida que N cresce, o buffer absorve melhor as variações de escalonamento: em $N = 1$ a ocupação alterna rigidamente entre 0 e 1; em $N = 10$ oscila amplamente; em $N = 100$ exibe rajadas intercaladas com quedas a 0; e em $N = 1000$ acumula centenas de itens antes de ser drenado.



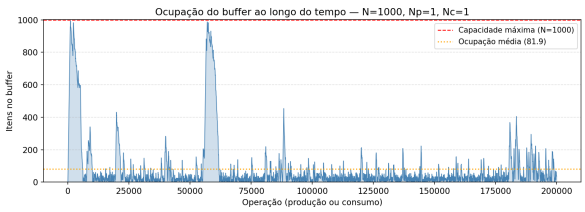
$N = 1$ (média 1,0)



$N = 10$ (média 4,7)



$N = 100$ (média 27,4)



$N = 1000$ (média 81,9)

Figura 3: Ocupação do buffer ao longo do tempo para $N_p = 1$, $N_c = 1$ e diferentes valores de N.

- **Impacto de N:** o tamanho do buffer é crítico apenas em $N = 1$, onde a serialização elimina o paralelismo e eleva o tempo de execução para até 6,9 s. A partir de $N = 10$, o ganho adicional de aumentar N é marginal.
- **Balanceamento N_p e N_c:** a combinação $(2, 1)$ foi a mais eficiente para $N \geq 10$. Assimetrias extremas como $(1, 8)$ ou $(8, 1)$ aumentam a contenção nos semáforos sem acelerar o processamento.
- **Eficácia dos semáforos:** o uso de semáforos contadores elimina o *busy wait*: threads bloqueadas não consomem CPU enquanto aguardam, ao contrário de spinlocks.
- **Variabilidade:** cenários com $N = 1$ e muitos produtores apresentaram alto desvio padrão, evidenciando dificuldade do escalonador em gerenciar várias threads competindo por um único slot de buffer.

2.5 Conclusões

A utilização de pipes como mecanismo de comunicação entre processos demonstrou-se simples e eficaz para o problema proposto. Por meio de um fluxo contínuo de bytes, foi possível garantir a troca ordenada de dados entre produtor e consumidor sem a necessidade de estruturas mais complexas. A padronização das mensagens em tamanho fixo evitou inconsistências de leitura, enquanto o bloqueio implícito do pipe forneceu uma forma natural de sincronização entre os processos, dispensando mecanismos adicionais.

Já nos experimentos com semáforos, observa-se que N tem impacto crítico apenas em valores muito pequenos ($N = 1$), onde a serialização elimina o paralelismo. A partir de $N = 10$, o desempenho passa a ser determinado principalmente pelo balanceamento entre N_p e N_c: a combinação $(2, 1)$ apresentou os melhores tempos, pois mantém o buffer abastecido sem gerar contenção excessiva nos semáforos.

O uso de semáforos contadores mostrou-se eficaz para coordenar o acesso sem *busy wait*: threads bloqueadas não consomem CPU enquanto aguardam condição favorável. O *overhead* de sincronização torna-se visível apenas quando muitas threads competem pelo mesmo recurso, como em $(8, 1)$ com $N = 1$, onde o elevado desvio padrão evidencia a dificuldade do escalonador em gerenciar 8 produtores disputando um único slot de buffer.